Ed25519 Signature Verification: Hardware Accelerator

Part of SentinelSoC, RISC-V based System-on-Chip for secure IoT applications

Overview

This module implements a complete RTL hardware accelerator for **Ed25519 digital signature verification** (RFC 8032). It is the cryptographic root-of-trust in SentinelSoC, authenticating firmware before the RISC-V core is permitted to execute it.

Key features:

- Full Ed25519 verification pipeline in hardware
- Embedded SHA-512 engine (NIST-compliant)
- Pseudo-Mersenne modular reduction for field arithmetic (mod $p = 2^{255} - 19$)
- Barrett reduction for group-order scalar arithmetic (mod $l \approx 2^{252}$)
- Shared iterative 256×256-bit multiplier — area-efficient, single instance
- Microsequencer-driven control — all point operations run through a programmable micro-op ROM
- AXI-Lite memory-mapped interface for integration with the RV32IMC core
- Verified against RFC 8032 official test vectors
- Prototype synthesis: 13,707 LUTs, 5,760 FFs (~21.6% of Artix-7 LUT budget)
- Timing closure: WNS +8.233 ns, all constraints met

How Ed25519 Works

The Curve

Ed25519 operates over the **twisted Edwards curve:**

$$-x^2 + y^2 = 1 + d \cdot x^2 \cdot y^2 \quad \text{over } GF(p), \quad p = 2^{255} - 19$$

where $d = -21665/55440 \bmod p$. This curve is birationally equivalent to Curve25519 (a Montgomery curve), giving it the same strong security properties while enabling highly efficient, branch-free point arithmetic.

Points on this curve form a group of order $8 \cdot l$, where $l \approx 2^{252}$ is a large prime used for all scalar arithmetic.

Why this curve for a hardware implementation?

- **128-bit security** with compact 32-byte public keys and 64-byte signatures

- The **complete twisted Edwards addition law** has no special cases — no conditional branches during point operations, eliminating an entire class of timing side-channel attacks by construction
- **Deterministic nonce generation** removes any dependency on hardware entropy sources
- These properties make it well-suited to strict area, power, and timing constraints

Keys and Signatures

Item	Size	Description
Public Key A	32 bytes	Compressed EC point (y-coordinate + sign bit of x)
Signature R	32 bytes	Compressed EC point
Signature S	32 bytes	255-bit scalar (little-endian)
Message M	variable	Arbitrary-length firmware blob

Private keys are never handled by this module — only the public key, loaded from OTP memory.

Verification Algorithm

Verification takes `(A, M, (R, S))` as input and runs four steps (RFC 8032, §5.1.7):

Step 1 — Hash:

```
h = SHA-512(R ‖ A ‖ M) mod l
```

Concatenate R, A, and M, hash with SHA-512, then reduce the 512-bit result modulo the group order `l` using Barrett reduction. This scalar `h` binds the signature to the specific message.

Step 2 — First scalar multiplication:

```
P1 = S · G
```

Scalar multiplication of the fixed generator point `G` by `S`. Uses 256 iterations of double-and-add — one doubling per bit, one conditional addition per 1-bit of `S`.

Step 3 — Decompress and second scalar multiplication:

```
P2 = R + h · A
```

Decompress `R` and `A` from their 32-byte forms by solving the curve equation for `x` given `y`. Compute `h · A` (another 256-iteration scalar multiplication) and add `R` to the result.

Step 4 — Cofactor check and equality:

```
8·P1 == 8·P2  ?
```

Multiply both points by the cofactor 8 (three doublings each) to clear torsion components, then compare in projective form:

```
(X₁·Z₂ == X₂·Z₁)  AND  (Y₁·Z₂ == Y₂·Z₁)
```

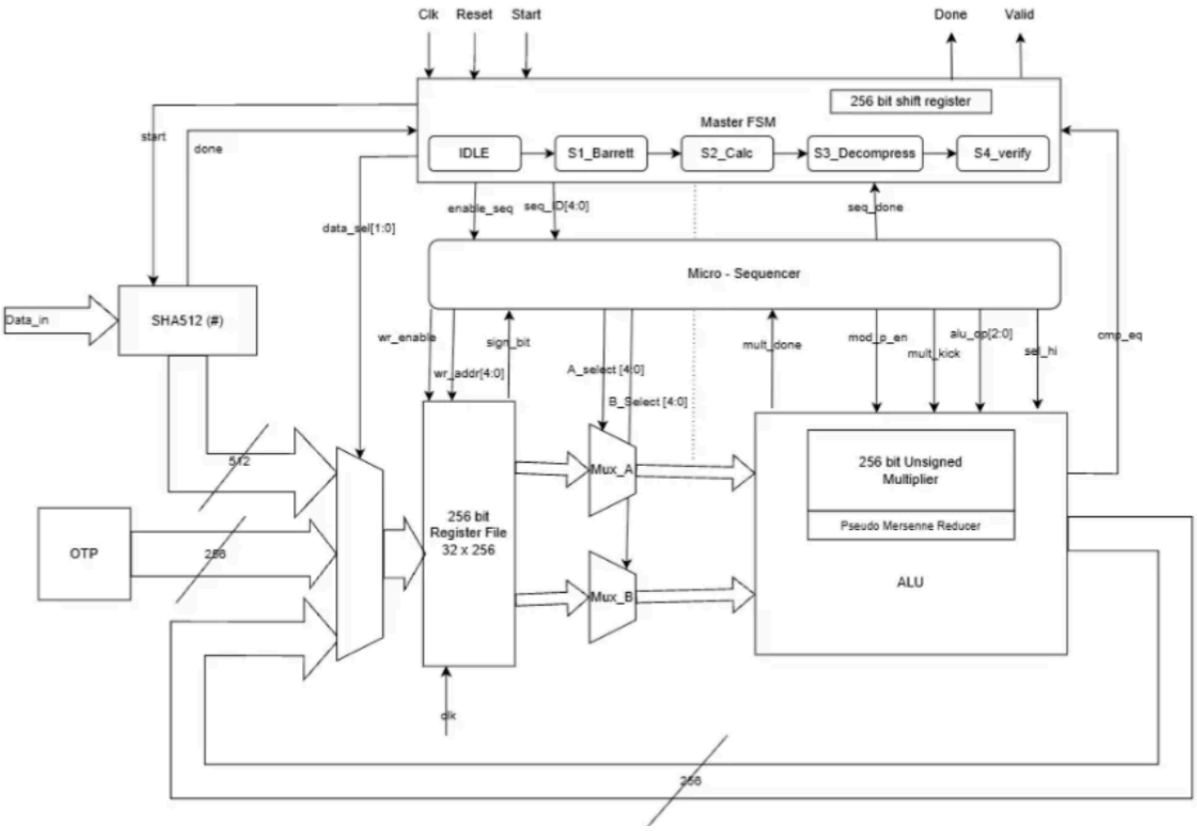
If both equalities hold → `signature_valid = 1`. Otherwise → halt.

Every point doubling and addition step internally runs a chain of **field multiplications modulo p**, each followed by Pseudo-Mersenne reduction to keep operands in range.

Hardware Architecture

Top-Level Design

The accelerator is partitioned into a **hashing front-end** and a **shared arithmetic datapath**, controlled by a two-level hierarchy (Master FSM → Microsequencer).



SHA-512 Engine

Role: Computes `h = SHA-512(R ‖ A ‖ M)` — the first step of verification. The 512-bit digest feeds directly into the Barrett reducer.

How it works:

Input arrives as a stream of 32-bit words. The engine handles little-to-big-endian conversion and message padding internally (appending a 1 bit, zero-padding, and a 128-bit length field to form complete 1024-bit blocks).

For each 1024-bit block:

- 80 message schedule words `W[0..79]` are generated. Rather than storing all 80, a **sliding window of 16 registers** is used: the oldest word is discarded each round and a new word is computed as `W[new] = σ₁(W[14]) + W[9] + σ₀(W[1]) + W[0]` via a Carry-Save Adder on the critical path.
- The compression function maintains 8 state registers `{a..h}` and runs 80 rounds:

```
t₁ = h + Σ₁(e) + Ch(e,f,g) + K[t] + W[t]
t₂ = Σ₀(a) + Maj(a,b,c)
```

where `Σ` functions are fixed rotations/XOR combinations, and `K[t]` are 80 pre-stored 64-bit round constants (ROM, 640 bytes).

Control FSM: Three states — `IDLE → WORK → ACCUM`. WORK runs 80 rounds; ACCUM adds the round output to the running hash H; IDLE signals completion via `intr_o`.

Output: 512-bit digest + `done` signal to the parent Ed25519 block.

Pseudo-Mersenne Reducer

Role: Reduces field arithmetic results (point coordinates) modulo $p = 2^{255} - 19$. Invoked after every field multiplication in the point arithmetic datapath.

How it works:

The prime $p = 2^{255} - 19$ is a pseudo-Mersenne prime, meaning $2^{255} \equiv 19 \pmod{p}$. This identity makes reduction very cheap compared to a general-purpose divider.

Given a 512-bit product A:

```
Split:   A = A_hi · 2²⁵⁵ + A_lo      (A_hi = 257 bits, A_lo = 255 bits)
Reduce:  r = A_lo + 19 · A_hi
```

The multiply-by-19 requires only a shift-and-add (19 = 16 + 2 + 1), yielding a 261-bit intermediate. At most two further split-and-reduce iterations are needed. A final conditional subtraction of `p` gives the fully reduced result in [0, p).

Hardware: 256-bit adder + 19× constant multiplier (shift-add) + carry chain + comparator with conditional subtractor.

This is substantially cheaper than Barrett or Montgomery reduction for this specific prime, making it the right choice for the high-frequency inner loop of point arithmetic.

Barrett Reducer

Role: Reduces the 512-bit SHA-512 hash to a scalar `h mod l`, where $l \approx 2^{252}$ is the group order. Used only in Step 1 of verification.

Why not Pseudo-Mersenne? `l` is not a Mersenne-type prime, so the identity trick doesn't apply. A general method is needed.

How it works:

Barrett reduction replaces division by l with a multiplication by a pre-computed reciprocal $\mu = \lfloor 2^{512} / l \rfloor \approx 261$ bits, stored as a constant in the design.

Given input A (up to 512 bits):

1. **Quotient estimate:** $q_0 = \lfloor (A \cdot \mu) >> 512 \rfloor$ — take the high half of the 512-bit product of A and μ
2. **Remainder candidate:** $r = A - q_0 \cdot l$
3. **Correction:** subtract l at most twice until $r \in [0, l)$

Both multiplications reuse the **shared iterative multiplier**, making the Barrett reducer primarily a control-flow and register-management concern rather than a significant area addition. Dominant cost: two full multiplications at 18 cycles each, plus comparison and subtraction cycles.

256×256-bit Multiplier

Role: The single shared compute resource for all multiplications — field multiplications in point arithmetic and both multiplications inside Barrett reduction. Sharing one instance across all operations is the primary area-saving strategy of the design.

How it works — iterative schoolbook decomposition:

- Each 256-bit operand is split into four 64-bit words
- A **single 64×64-bit multiplier instance** computes 16 partial products sequentially
- Partial products accumulate into a 512-bit register via a pipelined accumulator (one pipeline register breaks the critical path between multiplier output and adder)
- Alignment of each 128-bit partial product uses a mux-based placement unit (avoids a barrel shifter)

FSM: `IDLE → CALC → DONE`, **fixed latency: 18 clock cycles** (1 setup + 16 multiply-accumulate + 1 flush).

The 64×64-bit core maps to **DSP48E1 primitives** on Artix-7 during synthesis. Asserting `start` aborts and restarts computation; the microsequencer ensures this never happens mid-sequence.

ALU and Register File

Register File: Synchronous 16×256-bit structure with two read ports and one write port, all driven by the microsequencer. Holds all intermediate state across the verification sequence: point coordinates (X, Y, Z, T) in extended projective form, scalar fragments, hash outputs, and temporaries. A dedicated `sign_bit` register captures the sign bit during public key decompression.

ALU: Supports 256-bit addition and subtraction, selected by a 2-bit `alu_op` signal from the microsequencer. Used for Pseudo-Mersenne and Barrett correction steps, coordinate accumulation, and the final projective comparison.

Input Muxes: `Mux_A` and `Mux_B` select between register file outputs and direct external inputs (SHA-512 hash, OTP public key) before feeding the multiplier and ALU operand ports.

Microsequencer

Role: Translates high-level operations (point doubling, point addition, decompression, Barrett reduction) into cycle-by-cycle control signals for the register file, ALU, and multiplexers. This is what allows the single shared multiplier-reducer datapath to execute the full Ed25519 protocol.

How it works:

A two-level ROM (`MICRO_ROM`) indexed by (sequence_id, step). Each entry is a 7-field microinstruction:

Field	Width	Purpose
a_sel	4 bits	Source A register address
b_sel	4 bits	Source B register address
alu_op	3 bits	Operation: add / sub / multiply / compare / pass / load-compressed / conditional-negate
sel_hi	1 bit	Select high or low half of multiplier output
dest_sel	4 bits	Destination register address
mod_p_en	1 bit	Route multiplier output through Pseudo-Mersenne reducer
is_last	1 bit	End of sequence — return status to Master FSM

A simple **linear step counter** (not a full program counter) advances through the ROM. `is_last` halts execution and returns flags to the caller.

Sequences implemented:

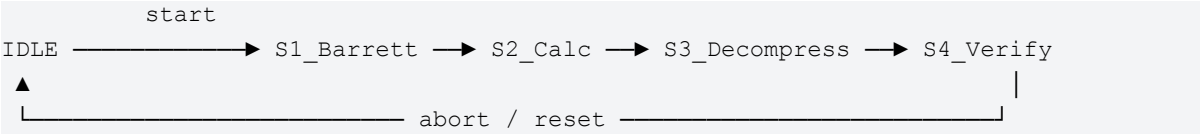
Seq ID	Operation	Steps
0	Barrett reduction of SHA-512 hash	—
1	Point doubling	14 steps
2	Point addition	18 steps
6–17	Point decompression (modular inversion, √, sign correction)	—
21–22	Projective cross-multiply equality check	—

Scalar multiplication is a loop in the Master FSM: sequence 1 (doubling) fires every iteration unconditionally; sequence 2 (addition) fires conditionally on each bit of the scalar.

Master FSM

Role: Top-level coordinator of the four verification phases. It schedules SHA-512, dispatches microsequencer sequences, and produces the final Done and Valid outputs.

State transitions:



State	Action
IDLE	Awaits start pulse; launches SHA-512 (runs autonomously)
S1_Barrett	On SHA-512 done interrupt: Barrett-reduce 512-bit hash → scalar h mod l
S2_Calc	Two scalar multiplications: S·G (→ P1) and h·A (→ intermediate for P2)
S3_Decompress	Decompress stored points R and A; complete point addition → P2
S4_Verify	Multiply P1 and P2 by cofactor 8 (3 doublings each); projective equality check → Valid

On any abort condition, the FSM asserts reset to the microsequencer and returns to IDLE.

Results of Verification and Testing on FPGA

The module was verified at two levels and prototyped on an FPGA to validate synthesis and timing ahead of SoC integration.

Algorithm-level verification: A complete Python model of the Ed25519 flow (Barrett reduction, field arithmetic, point operations, decompression) was validated against **RFC 8032 Test Vector 3**, producing the expected PASS result.

RTL-level verification: The SHA-512 module was verified in Vivado simulation against a null-input test vector. The full accelerator testbench confirmed correct FSM state transitions (`IDLE → S1_Barrett → S2_Calc → S3_Decompress → S4_Verify`) and correct assertion of `signature_valid`.

Prototype platform: Xilinx Artix-7 `xc7a100tcsg324-1` (Digilent Nexys 4), using Vivado 2025.2 for synthesis, place-and-route, and timing analysis. RTL is written in SystemVerilog.

Resource Utilisation

Resource	Used	Available (Artix-7)	Utilisation
Slice LUTs	13,707	63,400	~21.6%
Slice Registers	5,760	126,800	~4.5%

The shared-multiplier architecture is the primary contributor to the low LUT count.

Timing

Metric	Value
Worst Negative Slack (WNS)	+8.233 ns
Worst Hold Slack (WHS)	+0.029 ns
Failing endpoints	0 (of 16,134 checked)

All constraints met with positive slack.

Power

Dynamic power is dominated by the 512-bit accumulator in the multiplier and the carry chains of the 256-bit adders in the ALU and reducers, due to their high toggle rates during active computation. Full per-module breakdown is available in the Vivado power report.

References

4. R. Chaves et al., "Cost-Efficient SHA Hardware Accelerators," *IEEE TVLSI*, vol. 16, no. 8, pp. 999–1008, Aug. 2008.
5. D. J. Bernstein et al., "High-speed high-security signatures," *J. Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.
6. S. Josefsson and I. Liusvaara, "Edwards-curve Digital Signature Algorithm (EdDSA)," RFC 8032, Jan. 2017. https://tools.ietf.org/html/rfc8032
7. "High-speed hardware architecture design and implementation of Ed25519 signature verification algorithm," ResearchGate, Mar. 2026. https://www.researchgate.net/publication/402644364
8. M. Bisheh-Niasar et al., "Cryptographic Accelerators for Digital Signature Based on Ed25519," *IEEE TVLSI*, vol. 29, no. 7, pp. 1297–1305, July 2021.
9. C. Doche, "Low-Cost, Low-Power FPGA Implementation of ED25519 and CURVE25519 Point Multiplication," *Information*, vol. 10, no. 9, p. 285, Sept. 2019.